

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: Batch, Incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 9 - Orquestração, Jobs, DataOps e boas práticas

Orquestração, Jobs e DataOps

I&D Capgemini Brasil, Setembro/2026

ORQUESTRAÇÃO COM DATABRICKS JOBS



databricks DAY 9

ORQUESTRAÇÃO, JOBS, DATAOPS E MELHORES PRÁTICAS



CONFIÁVEL
E IDEMPOTENTE



ESCALÁVEL
E ELÁSTICO



OBSERVÁVEL
E GOVERNADO

FONTES DE DADOS



APIs / REST



BANCOS
DE DADOS



ARQUIVOS
EM CLOUD



STREAMING
(KAFKA)



APLICAÇÕES
SaaS



RETRIES & RECOVERY



Tentativas
automáticas com
backoff exponencial

ALERTAS & NOTIFICAÇÕES



Email, Slack, Webhook
e Microsoft Teams

MONITORAMENTO



Databricks UI, Logs,
Métricas e Dashboards

GOVERNANÇA & SEGURANÇA



Unity Catalog, ACLs,
Privilegios e Auditoria



DELTA LAKE

GIT & VERSIONAMENTO



- ✓ Notebooks versionados
- ✓ Code review (PR)
- ✓ Branches por ambiente
- ✓ Histórico e rastreabilidade



CI

- ✓ Lint
- ✓ Testes
- ✓ Validações
- ✓ Dependências



CD / DEPLOY

- ✓ Build
- ✓ Bundle/Jobs
- ✓ Deploy

DEV



Desenvolvimento
e testes

QA



Qualidade e
homologação

PROD



Produção
confiável

MELHORES PRÁTICAS

- ✓ Jobs pequenos e coesos
- ✓ Idempotência sempre
- ✓ Checkpoint dedicado
- ✓ Parâmetros e secrets
- ✓ Observabilidade completa
- ✓ Custos e performance



Objetivos do Dia 9

- Entender DAGs e orquestração de pipelines de dados.
- Criar um job com múltiplas tarefas no Databricks.
- Trabalhar com parâmetros, retries, notifications e dependências.
- Organizar notebooks/código com versionamento e modularidade.
- Aplicar DataOps: testes, documentação, CI/CD e observabilidade.

Mensagem central

Um pipeline útil precisa ser executável, auditável, reprocessável e monitorável.

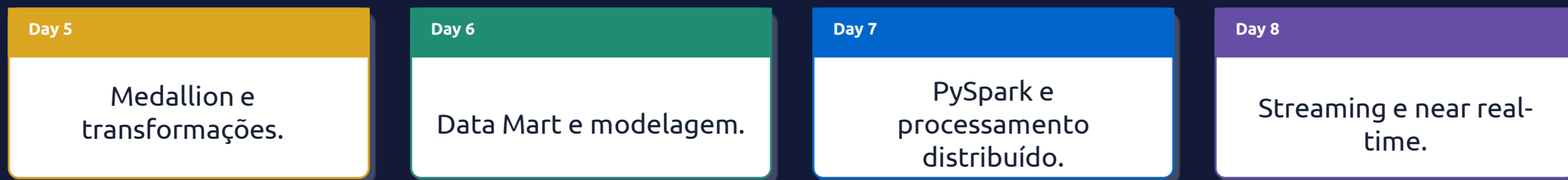
Entregável

Workflow operacional do pipeline completo, com teste básico de qualidade.



Recap: do dado ao pipeline operável

Os Dias 5 a 8 construímos lógica. O Dia 9 transforma lógica em operação controlada.



- Até aqui, os notebooks podem existir isolados.
- A partir de agora, o foco é executar, coordenar, validar e recuperar pipelines.
- O produto final do engenheiro é uma esteira confiável, não só um script correto.

Dia 9 - Orquestração, Jobs, DataOps e boias práticas

Bootcamp Databricks

I&D Capgemini Brasil



O que é orquestração?

Coordenar tarefas, dependências, parâmetros, estados e respostas a falhas.

Sequência

Executar tarefas na ordem correta.

Dependência

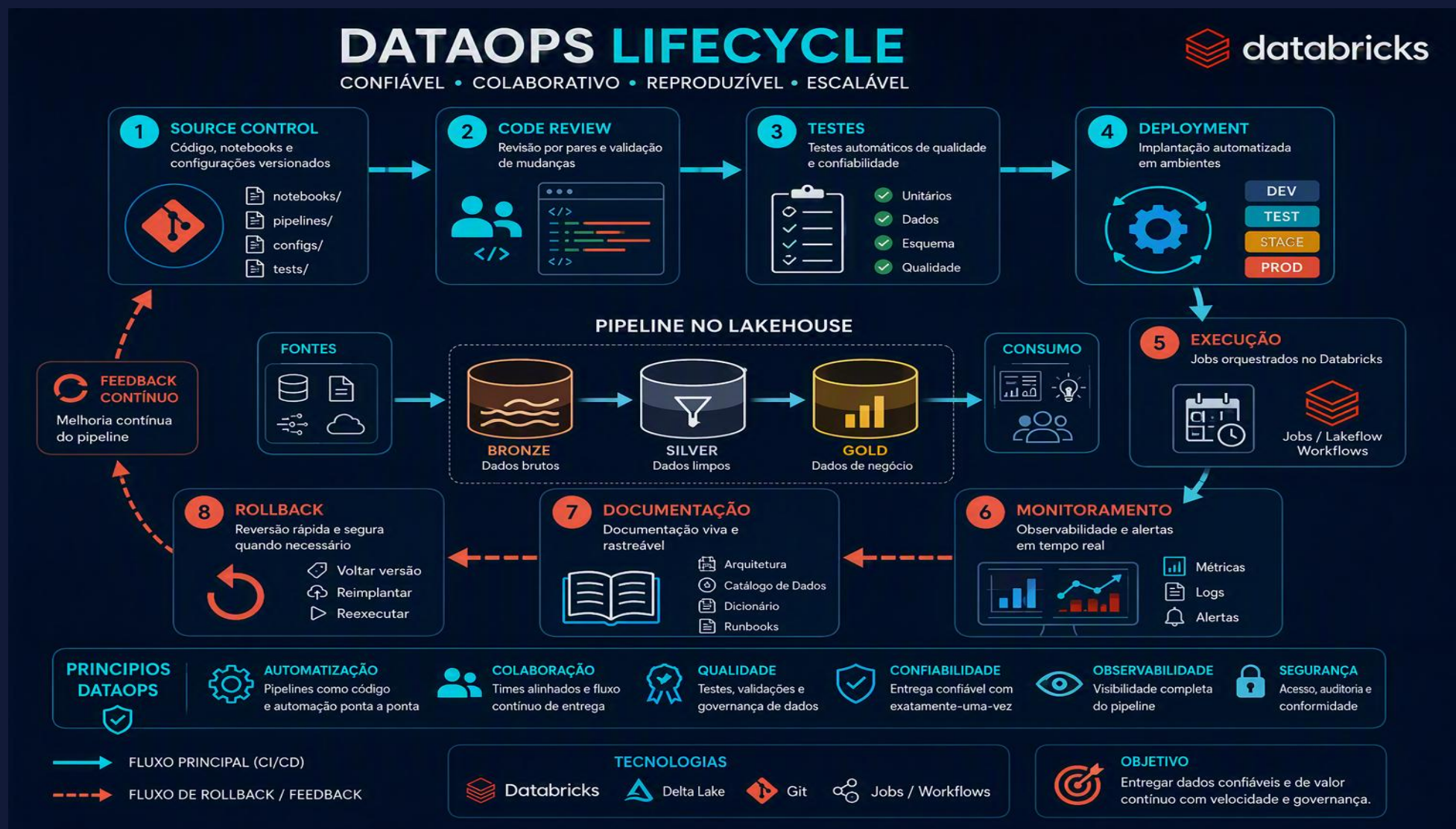
Só seguir quando pré-condições
forem atendidas.

Operação

Agendar, monitorar, alertar e
recuperar falhas.

- Orquestração não é apenas agenda.
- Um workflow expressa lógica operacional: ingestão, transformação, validação, publicação e tratamento de erro.
- A visualização em DAG ajuda a tornar dependências explícitas e governáveis.

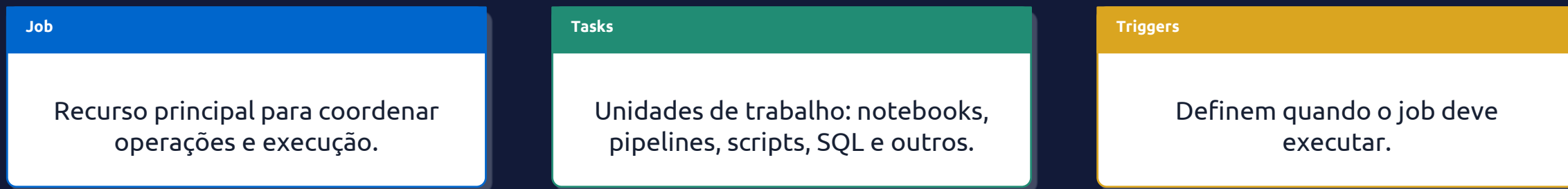
DAG de pipeline no Lakehouse





Lakeflow Jobs no Databricks

Jobs coordenam, agendam e executam tarefas como parte de um workflow.



- Jobs podem variar de tarefa única a workflows com muitas tarefas, lógica condicional e dependências.
- As tarefas de um job são representadas visualmente como DAG.
- Jobs suportam propriedades como parâmetros, notificações e Git para tarefas.



Componentes de um Job

O job combina lógica de negócio com configuração operacional.

COMPONENTE	PERGUNTA QUE RESPONDE
Job	Qual workflow será coordenado?
Task	Qual unidade de trabalho será executada?
Trigger	Quando ou em qual evento executar?
Parameters	Quais valores variam entre execuções ou ambientes?
Notifications	Quem ou o que será alertado em falhas ou atrasos?
Git	Qual versão de código será usada?



Desenho de workflow para o pipeline completo

O laboratório operacionaliza o pipeline dos dias anteriores.

TAREFA	FUNÇÃO	CAMADA
01_bronze_ingest	Ingerir novos dados	Landing → Bronze
02_silver_transform	Limpar, tipar e deduplicar	Bronze → Silver
03_quality_tests	Validar nulos, duplicidades e regras	Quality gate
04_gold_publish	Publicar agregados/modelo	Silver → Gold
05_notify_or_repair	Tratar falhas ou alertar	Operação



Parâmetros: reutilizar sem hardcode

Parâmetros tornam o workflow portátil entre ambiente, data e origem.

Ambiente

dev, qa, prd.

Data referência

job_run_date, load_date ou janela de processamento.

Origem/destino

source_path, target_catalog, schema e flags.

```
# Exemplo no notebook
dbutils.widgets.text('env', 'dev')
dbutils.widgets.text('target_catalog', 'dev_catalog')
dbutils.widgets.text('run_date', '')

env = dbutils.widgets.get('env')
target_catalog = dbutils.widgets.get('target_catalog')
```



Retries e recovery

Falhas transitórias devem ser tratadas sem transformar o pipeline em operação manual.

Retry

Nova tentativa automática para falhas transitórias.

Backoff

Intervalo entre tentativas para reduzir pressão no sistema.

Repair run

Reexecutar tarefas com falha ou etapas selecionadas.

- Retries não substituem correção de bug.
- Use retries para instabilidade externa, timeout transitório ou falha temporária de serviço.
- Falhas de qualidade não devem ser mascaradas por retry infinito.
- Registre quando e por que uma execução foi reparada.



Notificações e SLAs

Pipeline operacional precisa avisar falha, atraso ou comportamento fora do esperado.

Falha

Notificar quando tarefa ou job falhar.

Duração

Alertar quando execução passar do esperado.

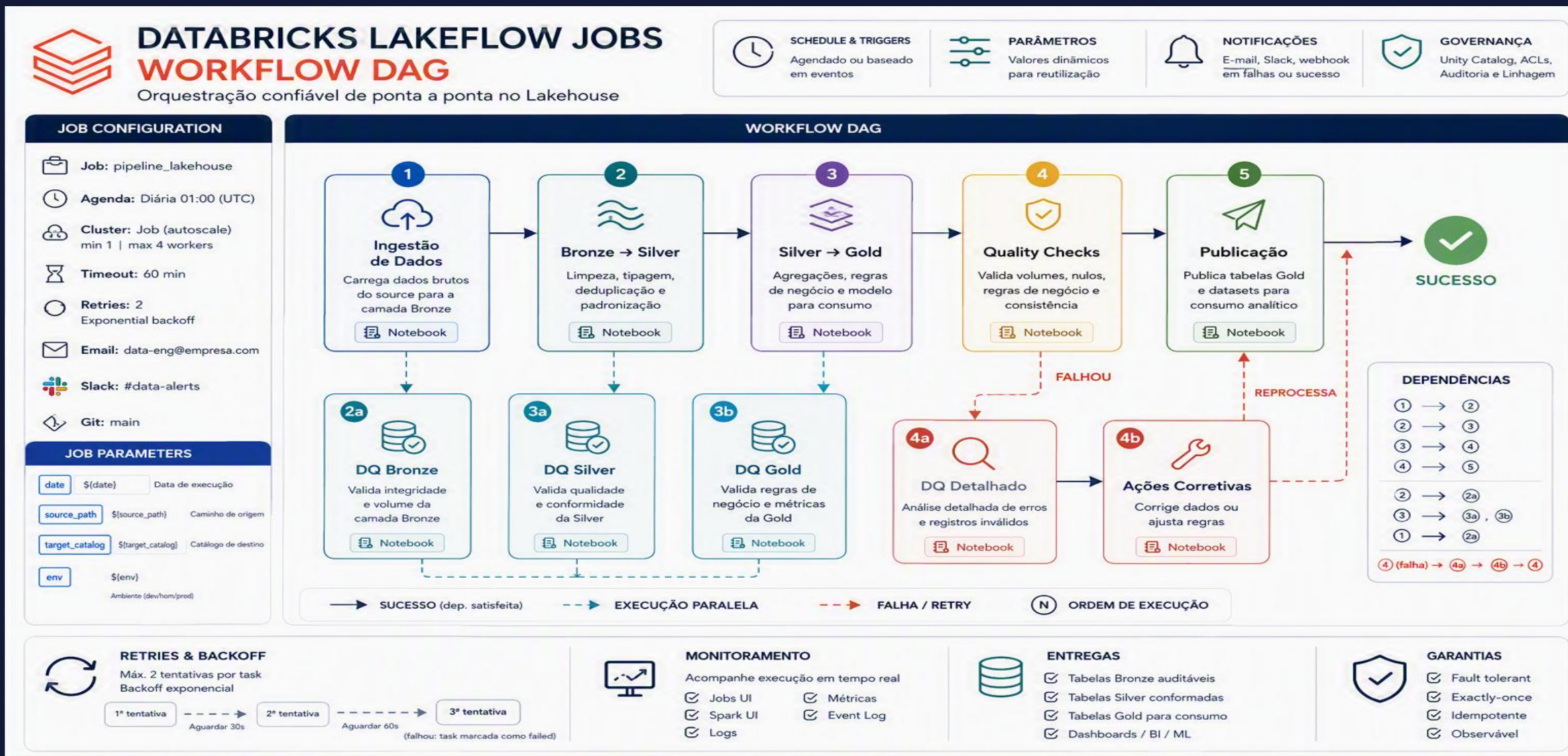
Observabilidade

Logs, métricas, owners e runbook.

- A notificação deve chegar ao responsável certo.
- Alertar demais cria ruído; alertar de menos gera falhas silenciosas.
- Todo alerta relevante deve apontar para investigação ou runbook.

DataOps: o pipeline como produto

DataOps conecta engenharia, qualidade, operação e entrega contínua.





Princípios DataOps aplicados ao Lakehouse

O foco é confiabilidade, colaboração e reprodutibilidade.

PRINCÍPIO	APLICAÇÃO PRÁTICA
Versionamento	Código, notebooks, configs e regras no Git
Modularidade	Notebooks ou módulos por responsabilidade
Testes	Unitários, integração e qualidade de dados
Documentação	README, owners, tabelas, parâmetros e runbooks
CI/CD	Promover de DEV para QA e PRD com gates
Observabilidade	Métricas, logs, alertas e histórico de execução



Estrutura de repositório

Organização mínima para trabalhar em equipe e evoluir o pipeline.

```
databricks-pipeline-lab/  
├── databricks.yml  
├── src/  
│   ├── notebooks/  
│   │   ├── 01_bronze_ingest.py  
│   │   ├── 02_silver_transform.py  
│   │   ├── 03_quality_tests.py  
│   │   └── 04_gold_publish.py  
│   └── utils/  
├── tests/  
│   └── test_quality_rules.py  
├── resources/  
│   └── jobs.yml  
└── README.md
```

- Separe notebooks por responsabilidade.
- Inclua testes e documentação no mesmo repositório.
- Configurações do job devem ser tratadas como artefato versionado.
- Evite código solto no workspace sem histórico.



Git folders no Databricks

Git folders permitem desenvolver notebooks e arquivos seguindo boas práticas de software.

Versionamento

Histórico de mudanças e colaboração.

Branches

Desenvolvimento, integração, staging e produção.

PR e review

Revisão antes de promover mudanças.

- Use Git para notebooks, arquivos e configurações.
- Cada desenvolvedor deve trabalhar em branch apropriada.
- Automação pode promover mudanças para folders de produção ou usar bundles para deployment.



CI/CD em contexto de dados

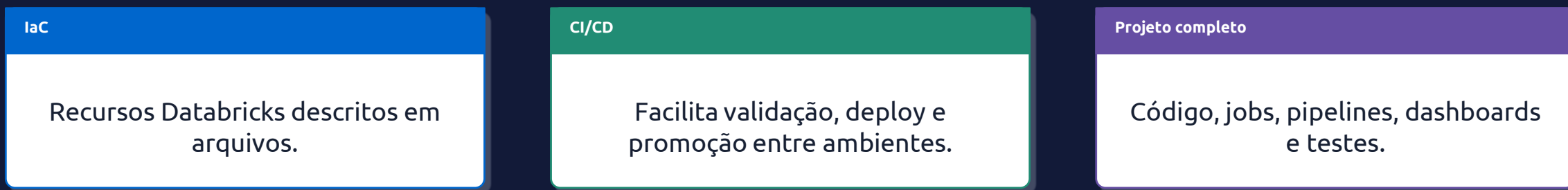
Promover código sem validar dados é incompleto. Dados exigem gates próprios.

ETAPA	EXEMPLO DE GATE
Commit / PR	Lint, revisão e teste unitário
Build	Resolver dependências e empacotar artefatos
Deploy DEV	Executar smoke test
Deploy QA	Testes integrados e qualidade de dados
Deploy PRD	Aprovação, monitoramento e rollback
Operação	Alertas e métricas pós-deploy



Declarative Automation Bundles

Bundles descrevem jobs, pipelines, código e testes como projeto versionável.



- Use bundles quando o projeto precisa de automação, múltiplos contribuidores e promoção controlada.
- Defina recursos como jobs e pipelines em source files.
- Diferencie targets: dev, qa/stg e prod.



Exemplo de bundle para job

Uma definição simples de workflow como código.

```
bundle:
  name: etl_medallion_pipeline

resources:
  jobs:
    etl_medallion_pipeline:
      name: ETL Medallion Pipeline
      tasks:
        - task_key: bronze_ingest
          notebook_task:
            notebook_path: ../src/notebooks/01_bronze_ingest.py
        - task_key: silver_transform
          depends_on:
            - task_key: bronze_ingest
          notebook_task:
            notebook_path: ../src/notebooks/02_silver_transform.py
```



Qualidade de dados como gate

A publicação deve depender de validações claras e auditáveis.

Schema

Colunas obrigatórias e tipos esperados.

Integridade

Nulos, duplicidades e chaves órfãs.

Regra de negócio

Métricas, limites, consistência e reconciliação.

```
invalid = spark.sql('''
SELECT COUNT(*) AS c
FROM ${target_catalog}.silver.orders
WHERE order_id IS NULL OR amount < 0
''').collect()[0]['c']

assert invalid == 0, f'Falha de qualidade: {invalid} registros inválidos'
```



Idempotência e reproprocessamento

Idempotência

Reexecutar produz o mesmo estado final esperado.

Reprocessamento

Recrutar períodos, partições ou chaves com controle.

Rollback

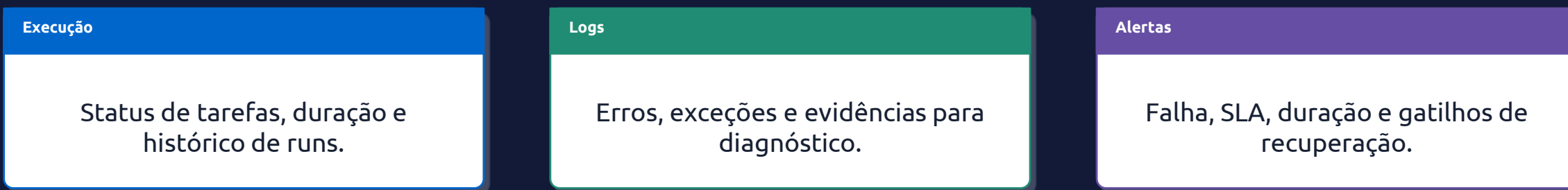
Voltar versão ou restaurar estado conhecido.

- Evite append cego em tabelas com possibilidade de reproprocessamento.
- Use chaves, partições e MERGE/overwrite controlado de forma explícita.
- Documente escopo de reproprocessamento e valide totais antes/depois.



Observabilidade de Jobs

O workflow precisa mostrar execução, falhas, duração, owner e lineage operacional.



- Sem observabilidade, uma falha vira surpresa.
- Métricas de execução ajudam a separar falha de dados, falha de código e falha de infraestrutura.
- Runbook deve orientar o que fazer quando o alerta dispara.

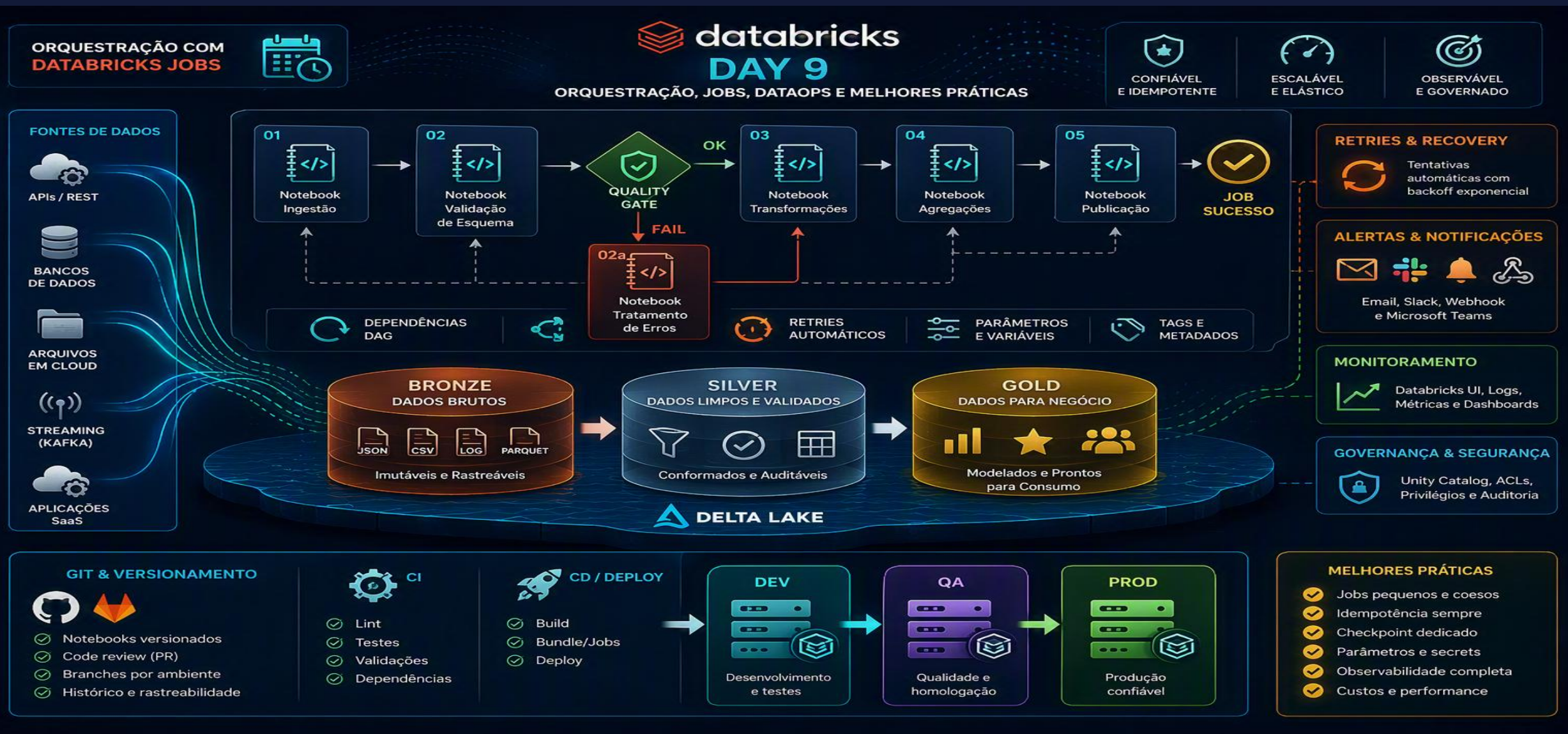


Runbook operacional

Todo pipeline crítico deve ter uma página curta de suporte.

CAMPO	CONTEÚDO ESPERADO
Owner	Time ou responsável pelo pipeline
Entrada	Tabelas, paths, fontes e frequência
Saída	Tabelas publicadas e consumidores
Parâmetros	env, run_date, reprocess, catalog
Falhas comuns	Causa provável e ação recomendada
Reprocessamento	Como reexecutar com segurança
SLA	Horário esperado, limites e alertas

Demo: criar Job com múltiplas tarefas

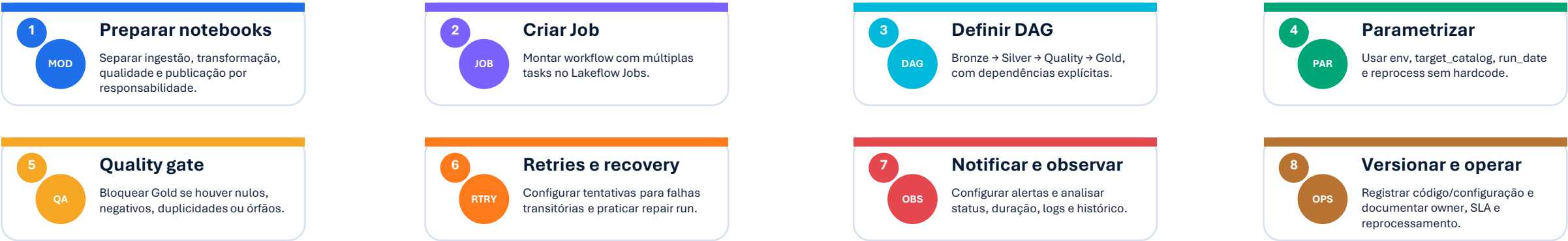


Objetivo: Transformar os notebooks e tabelas construídos nos Dias 4 a 8 em um workflow operacional com tarefas dependentes, parâmetros, quality gate, retries, notificações, versionamento e observabilidade.

Narrativa: os Dias 5 a 8 construíram lógica. O Dia 9 transforma essa lógica em uma esteira executável, auditável, reprocessável e monitorável.

Jornada prática do laboratório

Reutilize os notebooks de Bronze, Silver e Gold. Adicione qualidade e coordene tudo como DAG.



Sem operação, pipeline é experimento; com operação, vira produto.



Lab: workflow operacional do pipeline

O laboratório monta o pipeline completo como workflow executável.

PASSO	RESULTADO ESPERADO
1. Preparar notebooks	01_bronze_ingest, 02_silver_transform, 03_quality_tests, 04_gold_publish
2. Criar job	Workflow com tarefas e dependências
3. Adicionar parâmetros	env, catalog, run_date e reprocess
4. Configurar retry	Tentativas controladas para falhas transitórias
5. Adicionar teste básico	Bloquear publicação em caso de dado inválido
6. Versionar	Commit dos notebooks e configuração



Lab: notebook parametrizado

Parâmetros permitem executar o mesmo código em múltiplos ambientes.

```
dbutils.widgets.text('env', 'dev')
dbutils.widgets.text('target_catalog', 'dev_catalog')
dbutils.widgets.text('run_date', '')
dbutils.widgets.text('reprocess', 'false')

env = dbutils.widgets.get('env')
target_catalog = dbutils.widgets.get('target_catalog')
run_date = dbutils.widgets.get('run_date')
reprocess = dbutils.widgets.get('reprocess').lower() == 'true'
```

- Não deixar catálogo, schema ou data fixos no código.
- Parâmetros do job podem ser enviados para tarefas notebook.
- Padrão recomendado: ambiente + data + origem/destino + flags operacionais.



Lab: qualidade como etapa do workflow

Teste de qualidade vira tarefa própria, não comentário no final do notebook.

```
# 03_quality_tests.py
checks = {
    'null_order_id': spark.table(f'{catalog}.silver.orders').filter('order_id IS NULL').count(),
    'negative_amount': spark.table(f'{catalog}.silver.orders').filter('amount < 0').count()
}

failed = {k:v for k,v in checks.items() if v > 0}
if failed:
    raise Exception(f'Data quality failed: {failed}')
```

- Se quality_tests falhar, gold_publish não deve rodar.
- Falha de qualidade deve ser explícita e investigável.
- A etapa deve escrever evidências, quando possível.



Lab: checklist de configuração do Job

Configurações mínimas para deixar o workflow operável.

CONFIGURAÇÃO	STATUS ESPERADO
Nome	etl_medallion_pipeline
Tasks	Bronze, Silver, Quality, Gold
Dependencies	Silver depende de Bronze; Gold depende de Quality
Parameters	env, target_catalog, run_date, reprocess
Retry	Definido para falhas transitórias
Notifications	Falha ou duração fora do esperado
Git/Versionamento	Código referenciado por branch/commit controlado



Boas práticas de desenho operacional

Um pipeline confiável é pequeno, coeso, observável e reprocessável.

1. Modular

Uma tarefa, uma responsabilidade.

2. Parametrizado

Sem hardcode de ambiente e datas.

3. Validado

Quality gate antes de publicar.

- Evite notebooks gigantes com tudo misturado.
- Não esconda falhas com retries excessivos.
- Use logs e métricas para operar, não para enfeitar.
- Documente donos, entradas, saídas, parâmetros e reprocessamento.



Erros comuns no Dia 9

O workflow falha quando a operação é pensada tarde demais.

Erro 1

Criar job sem dono, alerta ou runbook.

Erro 2

Publicar Gold sem quality gate.

Erro 3

Deploy manual sem versionamento confiável.

- Job que só uma pessoa entende não é produto operável.
- Pipeline sem teste básico vira risco silencioso para consumidores.
- Configuração feita só na UI pode dificultar auditoria, revisão e promoção entre ambientes.



Fechamento do Day 9

O pipeline agora deixa de ser código isolado e vira produto operacional.

Conceito	Prática	Operação
DAGs, orquestração, Jobs e DataOps.	Workflow com múltiplas tarefas, parâmetros, retries e quality gate.	Versionamento, CI/CD, observabilidade e documentação.

- Entregável: workflow operacional do pipeline completo.
- O próximo passo natural é consolidar padrões, governança e operação para produção.
- Sem operação, pipeline é experimento. Com operação, pipeline vira produto de dados.

Fim do Day 9

Thank you.

Make it real.

Capgemini 